R2: A Narrow, On-Device Voice Assistant with Persistent Prefix KV Caching and Constrained Intent Decoding

Siavash Shokrtalab Full-stack engineering
Alireza Panahdoost Research and machine learning
Department of Computer Engineering, Islamic Azad University, North Tehran Branch, Tehran, Iran

Whitepaper, November 2026 (revision 2) · Code: `narrow-ai-with-kv` (MIT License)

Abstract

General-purpose assistants pay for their generality with large models, cloud round-trips and loose control over what they execute. We study the opposite design point: a *narrow* assistant that maps spoken or typed developer requests to one of eleven validated actions and runs entirely on a laptop. R2 combines a 4-bit Qwen3-1.7B model on Apple MLX, a *persistent prefix key–value (KV) cache* for its fixed 1,118-token instruction prompt, decoding constrained to the JSON schema of a decision, and a fail-closed execution layer in which model output is never authorization. We derive the exactness of prefix reuse for causal decoders, closed-form KV memory and a roofline-style cost model, and verify them on an Apple M4. Prefix caching reduces end-to-end time to first token (TTFT) from 1,460 ms to 71 ms (20.5×) with greedy outputs identical in 20/20 requests; loading the cache from disk costs 89 ms instead of a 1.5 s rebuild. A context-length sweep shows the forward-pass speed-up growing from 4.4× at 128 tokens to 77.9× at 4,096 tokens, matching a fitted cost model ($R^2 = 0.993$). On a fresh test set written before any prompt tuning, R2 selects the correct intent for 95.8% of requests (95% CI 79.8–99.3) with 0% malformed output. An ablation shows that few-shot examples drive accuracy; constraining only the intent name changes nothing, while a schema grammar recovers the 4–5 points that strict parameter validation would otherwise cost. With Whisper and Kokoro on device, the full voice loop answers in well under a second. We release the system, a SwiftUI client, benchmarks and notebooks.

Index Terms—KV cache, prefix caching, on-device LLM inference, constrained decoding, tool use, voice assistants, Apple Silicon, MLX.

1 Introduction

Voice assistants such as Siri answer in a fraction of a second because they solve a narrow problem with specialised models. Large language models (LLMs) promise a more flexible assistant but bring two costs that matter on a laptop: *latency*, because every request re-reads a long instruction prompt before producing a single token, and *control*, because a model that can emit any text can also emit an unintended command.

This whitepaper presents R2, a local, Siri-style assistant for software developers. R2 answers questions such as "what's my git status?", "find the Swift files" or "open Cursor" by classifying each request into one of eleven intents with typed parameters, validating it, asking for confirmation when the action changes something, and speaking a short reply. Its language model is small (1.7B parameters, 4-bit) and every component—speech recognition, the LLM, speech synthesis and the tools—runs on the user's Mac.

The key observation is that a narrow assistant's prompt is dominated by a *fixed prefix*: the instructions, intent catalogue and 25 worked examples occupy 1,118 tokens, while a typical request adds only about 22. Transformer decoders store per-token keys and values (the *KV cache*) that depend only on earlier tokens, so the prefix can be processed once and reused indefinitely. We make this cache *persistent*: it is written to disk with a fingerprint so that a restarted assistant is ready in tens of milliseconds.

Contributions.

1. A complete narrow-assistant architecture (section 5) with a fail-closed authorization model in which LLM output only *proposes* actions (section 6.3).
2. A formal treatment of prefix KV reuse: an exactness proposition for causal decoders, KV memory under grouped-query attention, and a cost model that predicts the speed-up bound (section 4).
3. A persistent prefix cache for MLX whose reuse is keyed on the *content* of the weights, tokenizer and adapter and validated layer by layer, and constrained decoders for the intent name and for the full decision schema, both failing closed (section 6).
4. Measurements on an Apple M4: end-to-end cache modes, a context-length sweep, memory, decode, disk I/O, accuracy with confidence intervals, an ablation, error analysis and voice latency (section 8).
5. An open implementation with a native macOS client, a terminal interface, reproducible benchmark scripts and teaching notebooks (section 11).

2 Background and Related Work

Attention and the KV cache. A decoder-only transformer [1] computes, in each layer, queries, keys and values $Q = XW_Q$, $K = XW_K$, $V = XW_V$ and causal attention $\mathrm{softmax}(QK^\top/\sqrt{d_h} + M)V$, where the mask M forbids at-

tending to future positions. During autoregressive generation each new token needs the keys and values of all previous tokens; storing them instead of recomputing them is the KV cache. Pope et al. [2] analyse its memory and latency trade-offs at scale and distinguish the compute-bound *prefill* phase from the memory-bound *decode* phase, the distinction we use in section 4.3. Multi-query [3] and grouped-query attention (GQA) [4] shrink the cache by sharing key/value heads; Qwen3-1.7B uses GQA with 16 query and 8 KV heads [5]. FlashAttention [6] makes the attention computation itself IO-efficient.

Reusing KV state across requests. PagedAttention [7] manages the cache in fixed-size blocks so that sequences can share memory, which serving systems such as vLLM build on to share common prefixes. SGLang's RadixAttention [8] keeps KV state of past requests in a radix tree with LRU eviction and reuses the longest matching prefix automatically. Prompt Cache [9] precomputes attention states for reusable *prompt modules* and reports 8–60× lower TTFT; because states depend on position, modules must reappear at the positions they were computed for. CacheBlend [10] relaxes the prefix requirement for retrieval-augmented generation by selectively recomputing a few tokens of each reused chunk, and CacheGen [11] compresses KV caches into bitstreams so they can be loaded from storage or the network. llama.cpp [12] can save and restore prompt state to files, and commercial APIs now expose prompt caching with time-limited server-side caches [13]. R2 occupies a simple point in this space: a *single, fixed* prefix on a *single-user* device, kept in RAM and on disk, with a content-derived fingerprint and a per-layer shape check that turn any stale or damaged file into a rebuild rather than a wrong answer.

Shrinking or evicting the cache. For long contexts the cache itself becomes the bottleneck. StreamingLLM [14] keeps attention-sink tokens plus a sliding window; H_2O [15] evicts all but "heavy-hitter" tokens; KIVI [16] and KVQuant [17] quantise keys and values to 2–4 bits. At R2's context lengths (about 1.2k tokens, 140 MiB) these are not needed, but they bound how far the design scales (section 9).

Structured output. Constrained decoding masks tokens that would violate a grammar. Outlines [18] compiles regular expressions and JSON schemas into finite-state machines over the vocabulary; grammar-constrained decoding [19] applies context-free grammars to structured NLP tasks; XGrammar [20] makes context-free constraints fast enough for serving; and LMQL [21] treats prompting as constrained programming. R2 first used the lightest form, a mask over the intent name only; it now compiles the whole decision schema to a grammar with llguidance [22] and still validates the result afterwards (section 4.5).

Tool use and assistants. Toolformer [23], ReAct [24] and Gorilla [25] teach LLMs to call tools or APIs. These systems maximise breadth; R2 deliberately minimises it, so that a 1.7B model reaches high accuracy through few-shot prompting [26] without fine-tuning, and every call passes a deterministic policy check.

Table 1: Prefix and KV reuse in related systems. "Persist" = survives a process restart.

System	Reuse unit	Where	Persist
PagedAttention / vLLM [7]	KV blocks	GPU memory	–
SGLang [8]	radix-tree prefixes	GPU/host	–
Prompt Cache [9]	prompt modules	GPU/CPU	–
CacheBlend [10]	any chunk + recompute	GPU/storage	✓
CacheGen [11]	compressed KV stream	network/disk	✓
llama.cpp [12]	whole prompt state	file	✓
Provider APIs [13]	marked prefixes	server, TTL	–
R2 (ours)	**fixed prefix, LRU**	**RAM (4) + disk (8)**	✓

On-device models. Post-training quantisation such as GPTQ [27] and AWQ [28] makes billion-parameter models practical on laptops. MLX [29] is an array framework for Apple Silicon's unified memory, and mlx-lm provides model implementations, quantised weights and prompt-cache utilities. For speech, R2 uses Whisper [30] for recognition and Kokoro-82M [31], a compact model in the StyleTTS 2 family [32], for synthesis.

Table 1 summarises how R2's cache compares with these systems.

3 Problem Formulation

Let $\mathcal{I}$ be a finite set of intents, here $|\mathcal{I}| = 11$: `git_status`, `git_diff_summary`, `search_files`, `read_file`, `open_app`, `remember_preference`, `recall_preference`, `answer_question`, `general_help`, `clarify_request` and `unsupported`. Each intent $i \in \mathcal{I}$ has a parameter schema Θ_i (for example $\Theta_{\texttt{search_files}}$ is a glob pattern). Given an utterance u and a short history h, the assistant must produce a *decision* $d = (i, \theta)$ with $\theta \in \Theta_i$, and then either execute it, ask for confirmation, ask for clarification or decline.

We optimise three objectives in priority order:

1. **Safety:** no action executes unless it is valid, allow-listed, inside the workspace and, for privileged actions, confirmed by the user (section 6.3).
2. **Accuracy:** the predicted intent and parameters match a reference.
3. **Latency:** minimise time to first token (TTFT) and total decision latency, since the intent name is fully determined within the first few generated tokens.

The language model p_ϕ defines d through its output string y. R2 restricts y either to start with the prefill $\pi =$ `{"intent": "` followed by a name in $\mathcal{I}$ (*intent* mode) or, by default, to the language of the decision schema (*json* mode).

4 Theoretical Analysis

4.1 Prefix invariance of causal decoders

Let $x_{1:N}$ be a token sequence, L the number of layers and $h_t^{(\ell)} \in \mathbb{R}^d$ the hidden state of token t after layer ℓ, with $h_t^{(0)} = E[x_t]$. With rotary position embeddings (RoPE) [33] applied at absolute position t, layer ℓ computes for each KV head g

$$k_t^{(\ell,g)} = \mathrm{RoPE}_t\Big(W_K^{(\ell,g)}\,\mathrm{norm}(h_t^{(\ell-1)})\Big), \tag{1}$$

$$v_t^{(\ell,g)} = W_V^{(\ell,g)}\,\mathrm{norm}(h_t^{(\ell-1)}), \tag{2}$$

and each query head j (mapped to KV head $g(j)$) attends only to positions $\le t$:

$$o_t^{(\ell,j)} = \sum_{\tau\le t} \frac{\exp\big(q_t^{(\ell,j)\top} k_\tau^{(\ell,g(j))}/\sqrt{d_h}\big)}{\sum_{\tau'\le t}\exp\big(q_t^{(\ell,j)\top} k_{\tau'}^{(\ell,g(j))}/\sqrt{d_h}\big)}\, v_\tau^{(\ell,g(j))}. \tag{3}$$

Proposition 1 (Prefix invariance)**.** *For every layer ℓ and position $t \le p$, the keys $k_t^{(\ell,\cdot)}$ and values $v_t^{(\ell,\cdot)}$ depend only on the prefix $x_{1:p}$ and not on any later token.*

Proof. By induction on ℓ. For $\ell = 1$, $k_t^{(1)}, v_t^{(1)}$ are functions of $h_t^{(0)} = E[x_t]$ and of t only. Assume that for layer ℓ every $h_\tau^{(\ell-1)}$ with $\tau \le t$ depends only on $x_{1:t}$. Then eqs. (1) and (2) make $k_t^{(\ell)}, v_t^{(\ell)}$ functions of $x_{1:t}$, and eq. (3) sums only over $\tau \le t$, so $o_t^{(\ell)}$ and hence $h_t^{(\ell)}$ (after the residual connection and MLP, which act position-wise) depend only on $x_{1:t}$. Taking $t \le p$ completes the induction. □

Corollary 1 (Exact reuse)**.** *Let $C(x_{1:p})$ be the per-layer keys and values of a prefix. For any suffix $x_{p+1:p+s}$, running the model on $x_{p+1:p+s}$ with cache $C(x_{1:p})$ yields, in exact arithmetic, the same logits as running it on $x_{1:p+s}$ from scratch, provided the suffix is placed at positions $p+1,\dots,p+s$.*

The positional proviso is why Prompt Cache must keep modules at fixed positions [9] and why CacheBlend needs recomputation for non-prefix chunks [10]. R2 always places its cached text at the start of the prompt, so the corollary applies directly. In floating point the two computations use different kernel shapes and summation orders, so logits can differ in low-order bits; section 8.4 measures this.

4.2 KV cache memory

Each cached token stores one key and one value vector per layer and KV head. With b bytes per element,

$$M(n) = 2\,L\,H_{\mathrm{kv}}\,d_h\,b\,n. \tag{4}$$

For Qwen3-1.7B ($L = 28$, $H_{\mathrm{kv}} = 8$, $d_h = 128$) in bfloat16 ($b = 2$) this is $2 \cdot 28 \cdot 8 \cdot 128 \cdot 2 = 114{,}688\,\mathrm{B} = 112\,\mathrm{KiB}$ per token. Without GQA (16 KV heads) the cache would be twice as large; the saving factor is $H/H_{\mathrm{kv}} = 2$. mlx-lm grows the cache in steps of $\kappa = 256$ tokens, so the allocated size is

$$M_{\mathrm{alloc}}(n) = 2\,L\,H_{\mathrm{kv}}\,d_h\,b\,\kappa\,\lceil n/\kappa\rceil\,. \tag{5}$$

R2's 1,118-token prefix therefore occupies $5 \cdot 256 = 1{,}280$ slots, i.e. 140 MiB (146.8 MB), which is exactly the size of the cache file R2 writes. For comparison, the 4-bit weights occupy about 0.94 GB, so the cache adds 15%.

4.3 A cost model for prefill and decode

Following the roofline view [2, 34], prefill of N tokens is compute-bound and decode is memory-bound. Let P be the number of parameters touched per token (including the language-model head) and H the number of query heads. A causal prefill costs

$$F_{\mathrm{pre}}(N) \approx \underbrace{2PN}_{\text{matmuls}} + \underbrace{2LHd_hN^2}_{\text{attention}} \tag{6}$$

FLOPs, where the attention term counts $QK^\top$ and AV ($2 \times 2LHd_h$ per query–key pair) over the $N^2/2$ causal pairs. With effective throughput Π and a fixed overhead a (kernel launches, synchronisation), the prefill time is

$$T_{\mathrm{pre}}(N) = a + bN + cN^2, \quad b = \frac{2P}{\Pi}, \quad c = \frac{2LHd_h}{\Pi}. \tag{7}$$

A *cached* prefill of s new tokens after a prefix of length n touches the weights s times and attends from s queries to $n+s$ keys:

$$T_{\mathrm{suf}}(s \mid n) = \alpha + bs + 4LHd_h\frac{s\,n + s(s+1)/2}{\Pi} \;\approx\; \alpha + \gamma n \tag{8}$$

for fixed s. Decoding one token with n cached tokens reads all weights W bytes and the cache,

$$T_{\mathrm{dec}}(n) = d_0 + d_1 n, \quad d_0 \approx \frac{W}{B}, \quad d_1 \approx \frac{2LH_{\mathrm{kv}}d_h b}{B}, \tag{9}$$

where B is the achieved memory bandwidth.

4.4 Speed-up of prefix caching

The TTFT speed-up of caching a length-n prefix for an s-token suffix is

$$S(n,s) = \frac{T_{\mathrm{pre}}(n+s)}{T_{\mathrm{suf}}(s \mid n)} = \frac{a + b(n+s) + c(n+s)^2}{\alpha + bs + O(sn)/\Pi}. \tag{10}$$

If overheads vanish ($a = \alpha = 0$) and attention is negligible ($c \to 0$), the speed-up is the token ratio

$$S^\star(n,s) = \frac{n+s}{s}, \tag{11}$$

which for R2 ($n = 1{,}118$, $s = 22$) is 51.8. Two effects push the measured value below $S^\star$: the fixed cost α of a forward pass, which dominates when s is small, and per-request work outside the model (tokenisation, sampling, constraint masks). Conversely, for long prefixes the quadratic term makes S grow faster than linearly. The crossover length at which attention costs as much as the matrix multiplications is $N^\star = b/c = P/(LHd_h)$.

Amortisation. Building the prefix costs one uncached prefill, $T_{\text{pre}}(n)$; each later request saves $\Delta = T_{\text{pre}}(n+s) - T_{\text{suf}}(s \mid n)$. In memory, caching pays for itself after $\lceil T_{\text{pre}}(n)/\Delta \rceil = 2$ requests. On disk, the build cost is replaced by a load cost $T_{\text{load}} \approx M_{\text{alloc}}(n)/B_{\text{disk}}$, paid once per process start.

4.5 Constrained intent decoding

Let $\mathcal{V}$ be the vocabulary, $\sigma(v)$ the string of token v, and $\mathcal{W} = \{w \,\|\, \text{"} : w \in \mathcal{I}\}$ the target strings (each intent name followed by its closing quote). After the prefill π, let $y_{<t}$ be the text generated so far. Write $a \preceq b$ if a is a prefix of b.

Definition 1 (Allowed set)**.**

$$A(y_{<t}) = \{v \in \mathcal{V} : \exists w \in \mathcal{W},\ y_{<t}\sigma(v) \preceq w \vee w \preceq y_{<t}\sigma(v)\}. \tag{12}$$

The decoder replaces the logits z_v by $z'_v = z_v$ for $v \in A(y_{<t})$ and $-\infty$ otherwise, so the sampling distribution becomes

$$p'(v \mid y_{<t}) = \frac{p(v \mid y_{<t})\,\mathbf{1}[v \in A(y_{<t})]}{\sum_{u \in A(y_{<t})} p(u \mid y_{<t})}. \tag{13}$$

Masking stops as soon as some $w \in \mathcal{W}$ is a prefix of the generated text.

Proposition 2 (Soundness)**.** *If $A(y_{<t}) \neq \emptyset$ at every masked step, the generated intent name is an element of $\mathcal{I}$.*

Proof. By construction every masked step keeps $y_{\le t}$ consistent with some target w, i.e. either a prefix of w or extending it. Since targets end with a quote that cannot occur inside a name, the first time a full target is a prefix of y the name equals some $w \in \mathcal{I}$, and masking stops. □

With greedy decoding, the choice becomes $\arg\max_{v \in A} z_v$, so masking can only change the output when the unconstrained argmax would have left $\mathcal{I}$. The mask is computed once per distinct prefix and memoised; candidate tokens are pre-filtered to those whose characters all occur in some target, so each step costs $O(|\mathcal{V}'|)$ with $|\mathcal{V}'| \ll |\mathcal{V}|$. If $A(y_{<t}) = \emptyset$ the precondition fails, so the decoder then allows only end-of-sequence tokens: the output fails to parse and takes the closed path of section 6.3, instead of continuing unconstrained.

Schema-constrained decisions. The intent mask says nothing about the parameters, which is where every malformed output in our evaluation occurred (section 8.7). In *json* mode the target language is $\mathcal{L}(G)$ for the grammar G compiled from the decision schema

$$S = \bigvee_{i \in \mathcal{I}_{\text{impl}}} \big(\texttt{intent} = i \ \wedge\ \texttt{parameters} \in \Theta_i\big), \tag{14}$$

an `anyOf` over implemented intents in which every object has `additionalProperties: false`. A matcher tracks the generated *bytes* and, at each step, yields the set $A_G(y_{<t})$ of tokens that keep y a prefix of some string in $\mathcal{L}(G)$. The matcher first consumes the prefill π; because matching is byte-level, any tokenisation of π is accepted.

Proposition 3 (Schema soundness)**.** *If $A_G(y_{<t}) \neq \emptyset$ at every step and generation ends when the matcher accepts, then $y \in \mathcal{L}(G)$; in particular y names an implemented intent and contains no parameter outside Θ_i.*

The argument is the one above with $\mathcal{W}$ replaced by $\mathcal{L}(G)$; an empty set or a matcher error again leaves only end-of-sequence. S deliberately constrains *structure*, not allow-listed *values*: an earlier version made `app_name` an enum of approved apps, and "open calculator" was decoded as `Terminal`, the closest legal string. A grammar always picks *some* legal value, so values that policy must refuse stay unconstrained and are rejected by the permission gate instead.

5 System Architecture

Figure 1 shows R2's components. A Python core holds all intelligence and policy; clients are thin. The core is reachable in-process (terminal app, notebooks) or over a local HTTP API bound to `127.0.0.1:8765` (SwiftUI app). The API refuses non-loopback binds and requires a bearer token on every route except `/health` (section 6.3).

5.1 Request lifecycle

Algorithm 1 gives the per-request control flow. The orchestrator keeps the last three user/assistant turns as compact JSON decisions rather than verbose tool output, which keeps the prompt short and stable.

5.2 Prompt design

The system prompt (fig. 2) contains 438 tokens of instructions—the JSON format, one line per implemented intent with its parameter schema and *when to use it*, the approved apps and boundary rules—followed by 25 few-shot examples (675 tokens, 61% of the prefix). The chat template adds 5 tokens, giving the 1,118-token cached prefix. Qwen3's thinking mode is disabled (`enable_thinking=False`) because the task needs a label, not reasoning, and thinking tokens would multiply latency.

5.3 Clients and voice

The SwiftUI client records audio with AVAudioEngine, uploads WAV to `/ask_audio`, and plays Kokoro speech from `/tts`, metering playback loudness at 60 Hz to animate an orb. A global `Option+Space` hot key, registered through Carbon's `RegisterEventHotKey`, summons R2 without Accessibility permissions. The terminal client offers push-to-talk and the same commands. The app launches the core with `uv run r2 serve` and stops it on quit. On start it reads the session token from a `0600` file and sends it with every request, rereading it once after a 401 (the core rotates the token on every restart). It stores the confirmation id from a prompt and attaches it both to the *Confirm* button and to the next spoken utterance, so a voice "yes" approves exactly the action that is on screen.

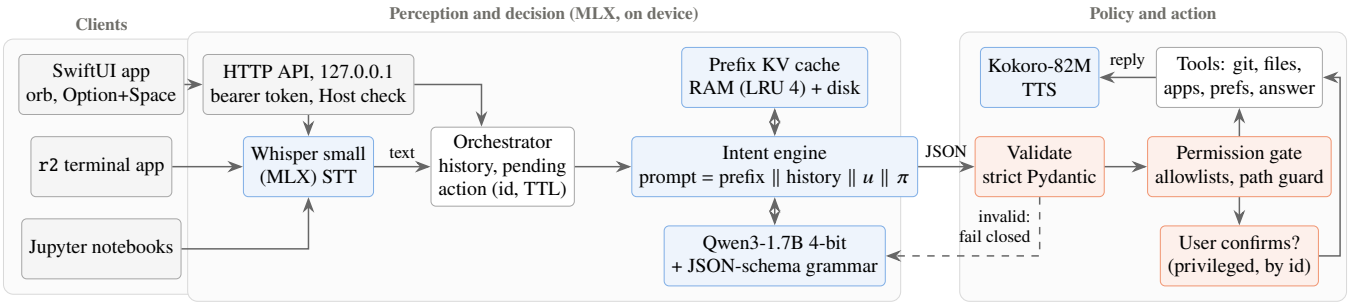


Figure 1: R2 architecture. Blue: learned models on MLX. Orange: deterministic policy that alone can authorise execution. The LLM only proposes a JSON decision; invalid proposals fall back to a safe heuristic, a clarifying question or a refusal.

Algorithm 1 Handling one request in R2

Require: utterance u (typed, or Whisper transcript), history h
Require: confirmation id c (empty unless the client is answering a prompt)
if a pending action P exists **then**
  if $u \in$ YES **then**
    $P \leftarrow$ TAKE(P) ▷ single use
    if P.CHECK(c) **then return** EXECUTE($P.d$) via the gate
    else return "Cancelled." ▷ missing, wrong, expired, altered
  else if $u \in$ NO **then**
    drop P; **return** "Cancelled."
  else drop P and handle u as a new request
  end if
end if
$x \leftarrow$ tokens(prefix ‖ h ‖ u ‖ π)
$(C, x_{\text{new}}) \leftarrow$ KV.ACQUIRE(x) ▷ reuse cached prefix
$y \leftarrow$ GENERATE(x_{new}, C, mask $= A_G(\cdot)$, greedy)
KV.RELEASE(C) ▷ trim back to the prefix
$d \leftarrow$ PARSEJSON($\pi \| y$); $d \leftarrow$ NORMALISE(d) ▷ unknown keys: reject
if d invalid **then**
  retry once, else heuristic / clarify / unsupported
end if
$r \leftarrow$ GATE(d), then the tool's precheck ▷ allowlist, schema, path guard
if r denied **then return** refusal
else if r needs confirmation **then**
  $P \leftarrow (d,$ id, TTL, sha256(d))
  return "Open Safari? Say yes or no." and the id
else return SPEAK(EXECUTE(d))
end if

5.4 Bounded multi-step requests

A request such as "check git status and then find the Swift files" becomes a *plan* of at most five ordinary decisions. The runtime adds sequencing but no new way to act: every step is handed to the same executor (gate, validation, precheck, confirmation, tool), so deleting the runtime would not change the set of reachable actions. The plan is made *once*, before any tool runs, by splitting the request on explicit connectors ("and then", "after that", ";") and routing each clause through the normal single-intent engine. A clause that routes to a tool action and contains "and" or a comma is split further only if

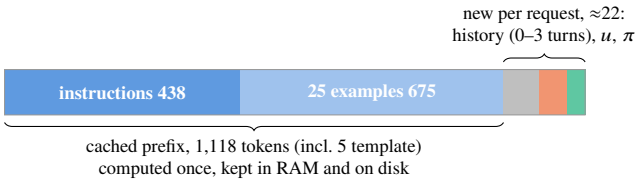


Figure 2: Token layout of one request. Only the right-hand tokens (grey: history, orange: request, green: prefill π) are processed per request; after generation the cache is trimmed back to the prefix.

every part also routes to a tool action, and a request whose split would be rejected is routed once more as a whole, so that "how do I rebase and then push?" is answered as one question. Tool output is never fed back to a planner, so a file that says "now open Terminal" cannot add a step, which is our main containment of prompt injection. The whole plan is rejected if any step is invalid, unimplemented, `unsupported` or `clarify_request`. Privileged steps pause the run with the same single-use, expiring confirmation id; the first failing step ends the run, and a wall-time budget bounds it. Run history is stored as metadata only (intents, parameters, status, the short spoken summary), never file contents or diffs. A one-shot model planner that emits the whole plan under a grammar is available but not the default (section 8.8).

The app reaches runs through the ordinary `/ask` endpoint with an `allow_runs` flag: a request that plans to two or more steps starts a run, and the reply carries the run so the client can draw one row per step. A paused run hands out its confirmation id exactly like a single action, so the Confirm button, a typed "yes" and a spoken "yes" all continue it, and a new request supersedes it. If any clause is refused, the reply is a refusal and nothing runs; the client never falls back to executing the parts it understood. The Confirm button requires Command–Return so that a stray Return can never approve a step.

6 Implementation

6.1 Persistent prefix KV cache

Listing 1 shows the essential logic of `PrefixKVCache`. Three modes support experiments: `none`, `prefix` (RAM) and

Listing 1: Acquire, release and fingerprinted persistence (abridged from `models/kv_cache.py`).

```
def fingerprint(**parts):  # revision, weights/tokenizer/config/adapter
    h = sha256(json.dumps(parts, sort_keys=True).encode())  # shas, template,
    return {**parts, "fingerprint": h.hexdigest()[:20]}     # mlx versions, prefix

def acquire(self, full_tokens):
    n = len(self._prefix_tokens)
    if (self._cache is None or len(full_tokens) <= n
            or full_tokens[:n] != self._prefix_tokens):
        return None, full_tokens, 0          # miss: full prefill
    return self._cache, full_tokens[n:], n   # hit: suffix only

def release(self, cache):                    # keep only the prefix
    extra = cache[0].offset - len(self._prefix_tokens)
    if extra > 0:
        trim_prompt_cache(cache, extra)

def _try_load(self, fp):
    cache, meta = load_prompt_cache(path, return_metadata=True)
    mx.eval([c.state for c in cache])        # loads are lazy
    if meta.get("fingerprint") != fp["fingerprint"]:
        return False                         # stale: rebuild
    return validate_layers(cache, self._layer_classes,
                           len(self._prefix_tokens)) is None

def _save(self, fp):                         # atomic, then evict LRU
    save_prompt_cache(tmp, self._cache, metadata=fp)
    os.replace(tmp, path)
```

`persistent` (RAM + disk). Several prefixes—the intent prompt and the answer prompt—share an LRU of four entries in memory; on disk, at most eight files are kept, evicted by least-recent use.

Because corollary 1 requires an identical prefix at identical positions *under identical weights*, the fingerprint is derived from content rather than names. In the Hugging Face cache, snapshot files are symbolic links to blobs named by the hash of their content (SHA-256 for weight files) [35], so reading the link targets gives the identity of the weights, tokenizer and configuration at no I/O cost. Files in a plain directory are hashed with SHA-256 up to 64 MiB and identified by size and modification time above that, which is weaker and documented as such. The fingerprint also covers the snapshot revision, the adapter files, a hash of the chat template, the mlx and mlx-lm versions and a hash of the prefix text; any change invalidates the file and triggers a rebuild, which is what makes persistence safe. A matching fingerprint is not trusted on its own: `validate_layers` checks the number of layers and each layer's cache class against the loaded model, and that *every* layer's offset and key length equal the prefix length. A truncated or mismatched file is therefore rebuilt rather than used. Files are written to a temporary name and renamed into place, so a crash cannot leave a torn cache. The load is followed by `mx.eval` because MLX loads arrays lazily; without it, load times would be under-reported. The token-level check in `acquire` also guards against tokeniser merges across the prefix boundary: if the rendered prompt does not start with exactly the cached tokens, R2 falls back to a full prefill.

Listing 2: Intent-name mask (abridged from `models/constrained.py`).

```
def allowed_ids(self, prefix):
    if prefix not in self._allowed_cache:
        self._allowed_cache[prefix] = [
            tid for tid, s in self._candidates
            if any(t.startswith(prefix + s) or (prefix + s).startswith(t)
                   for t in self.targets)]
    return self._allowed_cache[prefix]

def __call__(self, tokens, logits):
    if self.done:
        return logits
    text = self.tokenizer.decode(tokens[self.n_prompt:].tolist())
    if any(text.startswith(t) for t in self.targets):
        self.done = True                     # name complete
        return logits
    allowed = self.allowed_ids(text)
    if not allowed:                          # fail closed
        return eos_only(logits, self.eos)
    mask = np.full(logits.shape[-1], -np.inf, np.float32)
    mask[allowed] = 0.0
    return logits + mx.array(mask)
```

6.2 Constrained decoder

Listing 2 implements the allowed set of section 4.5 as an mlx-lm logits processor. The vocabulary strings and candidate list are cached at module level, so the cost is paid once per process.

Schema grammar. In json mode (the default) the decision schema of section 4.5 is generated from the Pydantic parameter models of the implemented intents and compiled by llguidance [22] with the separators used in the few-shot examples (`", "` and `": "`) and no free whitespace, so the grammar and the prompt agree on one canonical form. The compiled matcher is cached per tokenizer and schema and copied for each request. Per step the processor feeds new tokens to the matcher, fills a token bitmask on the CPU and applies it to the logits with a Metal kernel; a rejected token, a matcher error or an empty mask leaves only end-of-sequence. If json mode is configured but llguidance is missing, R2 refuses to start rather than silently falling back to a weaker mode.

6.3 Validation, normalisation and safety

Model output passes four deterministic stages before anything executes.

1. **Parsing:** the first complete JSON object is extracted with a non-greedy decoder.
2. **Normalisation:** unknown parameter keys are *rejected*, which sends the request to the retry and fallback path (an earlier version silently dropped them), and search terms are mapped to globs, e.g. `swift`, `*swift*` or "swift files" → `*.swift`, while `config` → `*config*`.
3. **Validation:** a Pydantic model per intent rejects missing or extra fields and over-long values.
4. **Authorisation:** the permission gate checks the intent is implemented and enabled, apps are on the allowlist, and paths resolve inside the workspace. A tool-specific

precheck (for example the app allowlist) runs *before* any confirmation is requested, so the user is never asked to approve an action that would be refused anyway.

Formally, a decision d executes only when

$$\mathrm{Exec}(d) \iff \mathrm{Valid}(d) \wedge \mathrm{Allowed}(d) \wedge \mathrm{InWorkspace}(d) \wedge \big(\neg \mathrm{Priv}(d) \vee \mathrm{Confirm}_{\mathrm{user}}(d)\big), \quad (15)$$

where $\mathrm{Confirm}_{\mathrm{user}}$ can only come from a later user turn, never from model text. It is also bound to the prompt the user saw: a pending action carries a random single-use id, a time-to-live (120 s) and a SHA-256 digest of the decision, and a "yes" approves it only if the id matches, the action has not expired and the digest is unchanged. Any other attempt cancels the action, and execution re-runs the gate. In-process clients (terminal, notebooks) show the prompt in the same process and do not need the id. Every failure path is closed: a parse error triggers one retry and then a heuristic classifier whose outputs are themselves validated; reading files refuses secrets (`.env*`, `*.pem`, `*.key`, `.netrc`); and `answer_question` produces text that is displayed and spoken but never executed. There is no shell tool.

File tools. Path checks on names alone are vulnerable to symbolic links and to a change between check and use. `read_file` opens with `O_NOFOLLOW`, refuses anything that is not a regular file (so a FIFO cannot block it), and re-checks the real path of the *opened descriptor* (`F_GETPATH` on macOS) against the workspace and the secret-file rules before reading. `search_files` walks the tree without following directory links, skips links that leave the workspace, and replaces control characters in file names so a crafted name cannot spoof the spoken or displayed output. Adversarial tests cover links to outside files, a link named `notes.txt` that points at `.env`, both race orders, FIFOs and spoofed names.

Local HTTP API. A loopback-only server is still reachable by any web page the user visits, and by DNS rebinding. Following the guidance for local MCP servers [36], every request must carry a `Host` of `127.0.0.1`, `localhost` or `[::1]` on the bound port; requests with an `Origin` header or `Sec-Fetch-Site: cross-site` are refused; bodies must be JSON (or WAV for audio), which forces a CORS preflight that the server never grants; and every route except `/health` needs a bearer token. The token is random, rotated on every start, written atomically to a `0600` file and removed on shutdown. Browsers are also adding permission prompts for requests to local addresses [37], but R2 does not depend on them. The boundary is deliberately the user account: a process running as the same user can read the token, as it can read the preference database and the configuration. The token stops browsers, other users and accidental clients, not malware the user is already running.

Table 2: Model configuration (Qwen3-1.7B, MLX 4-bit).

Layers L	28	Hidden size d	2,048
Query heads H	16	KV heads H_{kv}	8
Head dim d_h	128	FFN size	6,144
Vocabulary	151,936	Weights	4-bit, group 64
Weight file	0.94 GB	KV / token	112 KiB

6.4 Software

The core is Python 3.12 (managed by `uv`) with mlx 0.32.4, mlx-lm 0.32.2, mlx-whisper 0.4.3, mlx-audio 0.6.3, llguidance 1.9.1 and Pydantic 2.14; the client is SwiftUI (Swift 6.3, macOS 14+). 195 unit, integration and adversarial tests run without model downloads, a microphone or network access.

7 Experimental Setup

Hardware and model. All measurements use an Apple M4 (4 performance and 6 efficiency CPU cores, 8-core GPU) with 16 GB of unified memory and a nominal 120 GB/s memory bandwidth, on macOS 26.5. The model is `mlx-community/Qwen3-1.7B-4bit` (table 2). Decoding is greedy (temperature 0) with at most 128 new tokens.

Datasets. The *dev set* (`eval_heldout.jsonl`, 40 cases) covers ten intents and was used while designing the prompt. The *fresh test set* (`eval_fresh.jsonl`, 24 cases, all eleven intents) was written *before* the final round of prompt changes and never used to tune them; we report it both before and after that round. Neither set overlaps the few-shot examples.

Metrics. *Intent accuracy* is exact match of the intent. *Parameter accuracy* additionally requires the normalised required parameters to match. *Malformed rate* counts outputs that do not parse into a valid decision; *fallback rate* counts requests answered by the heuristic. TTFT is measured from the start of generation to the first token; *decision latency* covers prompt rendering, generation, parsing and validation. For proportions we report 95% Wilson score intervals.

Benchmarks. `bench_kv.py` runs 20 dev utterances end-to-end in each cache mode. `bench_scaling.py` isolates the model: for prefix lengths n from 128 to 4,096 tokens (R2's real prompt, repeated as needed) and a 22-token suffix it times an uncached forward pass over $n + s$ tokens, a cached forward pass over s tokens, one decode step, KV size, and safetensors save/load, reporting the median of 3 runs after warm-up. `bench_voice.py` times Whisper on 8 synthetic utterances and Kokoro on 4 replies.

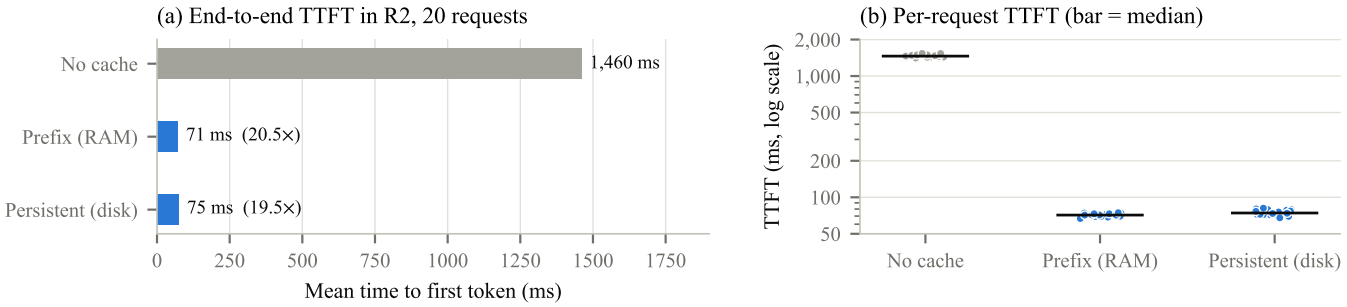


Figure 3: End-to-end TTFT inside R2 for 20 dev requests. Both cached modes reach about 71–75 ms; the spread across requests is small because only the short suffix varies.

Table 3: End-to-end cache modes, 20 requests (mean ± s.d.).

Mode	Setup	Prefilled	TTFT (ms)	Decision (ms)
None	–	1,140	1460 ± 41	1628 ± 82
Prefix (RAM)	1.53 s build	22	**71 ± 2**	227 ± 41
Persistent	89 ms load	22	75 ± 4	227 ± 42

Table 4: Model-only sweep, $s = 22$ (median of 3). KV is the allocated size; Δz is the largest absolute logit difference between cached and uncached runs.

n	Uncached (ms)	Cached (ms)	S	Decode (ms)	KV (MiB)	max $\lvert\Delta z\rvert$
128	249	56.5	4.4	14.2	28	0.47
256	433	59.0	7.3	14.1	28	0.38
512	806	60.9	13.2	14.7	56	0.31
1,024	1,583	57.0	27.8	14.0	112	0.50
1,118	**1,723**	**57.6**	**29.9**	12.4	140	0.44
1,536	2,414	62.7	38.5	16.8	168	0.44
2,048	2,664	64.7	41.2	15.3	224	0.00
3,072	4,851	71.9	67.4	16.7	336	0.28
4,096	6,220	79.8	77.9	19.9	448	0.00

8 Results

8.1 End-to-end cache modes

Table 3 and fig. 3 show the effect inside the running assistant. Prefix caching cuts mean TTFT from 1,460 ms to 71 ms, a 20.5× speed-up, and whole decisions from 1.63 s to 0.23 s. The persistent mode is within 4 ms of the in-memory mode while replacing a 1.53 s rebuild with an 89 ms load at start-up (17× faster readiness). The measured speed-up is below the token-ratio bound of 51.8 (eq. (11)) because of the fixed per-pass cost and request work outside the model, quantified next.

8.2 Scaling with context length

Table 4 and fig. 4 isolate the model. Uncached prefill grows almost linearly, from 249 ms at 128 tokens to 6.2 s at 4,096, while the cached 22-token pass stays between 57 and 80 ms. Least-squares fits give

$$T_{\text{pre}}(N) = 65.4 + 1.365\,N + 3.51{\times}10^{-5}N^2 \text{ ms} \quad (R^2 = 0.993), \tag{16}$$

$$T_{\text{suf}}(22 \mid n) = 54.8 + 0.00556\,n \text{ ms} \quad (R^2 = 0.897). \tag{17}$$

These coefficients agree with eq. (7). Counting $P \approx 1.72$B parameters per token (1.41B in the transformer blocks plus the 0.31B tied output head), the linear coefficient implies an effective throughput $\Pi = 2P/b \approx 2.5$ TFLOP/s. At that throughput eq. (7) predicts $c = 2LHd_h/\Pi = 4.5{\times}10^{-5}$ ms/token^2, within 25% of the fitted $3.5{\times}10^{-5}$. The crossover length $N^\star = b/c \approx 39{,}000$ tokens confirms that R2 operates deep in the linear, weight-bound regime, where the speed-up approaches the token ratio minus overhead. At R2's prompt length the model-only speed-up is 29.9×; the end-to-end 20.5× of table 3 additionally includes about 14 ms of tokenisation, sampling and masking per request.

8.3 Memory, decode and disk

Figure 5a confirms eqs. (4) and (5) to the byte: every measured cache size equals $114{,}688 \cdot 256\lceil n/256 \rceil$ bytes. Decoding (fig. 5b) costs $13.2 + 0.00135\,n$ ms per token. Reading the 0.94 GB of weights in 13.2 ms implies an achieved bandwidth of about 71 GB/s (59% of nominal), and at that bandwidth eq. (9) predicts a per-context-token slope of $114{,}688/71$ GB/s $= 0.0016$ ms, close to the fitted 0.00135. The end-to-end generation rate is about 77 tokens/s (median), and R2's peak memory during evaluation is 1.36 GB.

Disk loading is fast when the file is in the page cache (9 ms for R2's 140 MiB at 1,118 tokens, about 16 GB/s), and 89 ms on a cold start of the assistant, which includes reading from SSD, parsing safetensors metadata and verifying the fingerprint. Both are far below the 1.5–1.7 s cost of rebuilding the prefix.

8.4 Numerical equivalence

In exact arithmetic corollary 1 guarantees identical logits. In bfloat16 the cached and uncached computations use different kernel shapes, so the largest absolute logit difference at the last position ranges from 0 to 0.5 (table 4), while the argmax

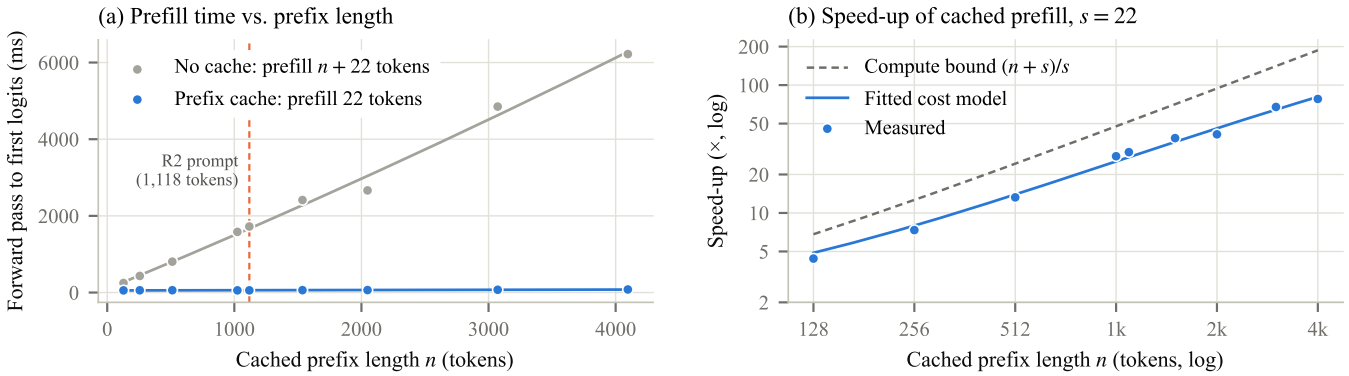


Figure 4: (a) Forward-pass time to the first logits for an uncached prompt of $n + 22$ tokens and for 22 tokens on top of an n-token cached prefix; lines are the fitted models of eqs. (7) and (8). (b) The resulting speed-up against the compute bound $(n + s)/s$; the gap is the fixed cost α of a forward pass.

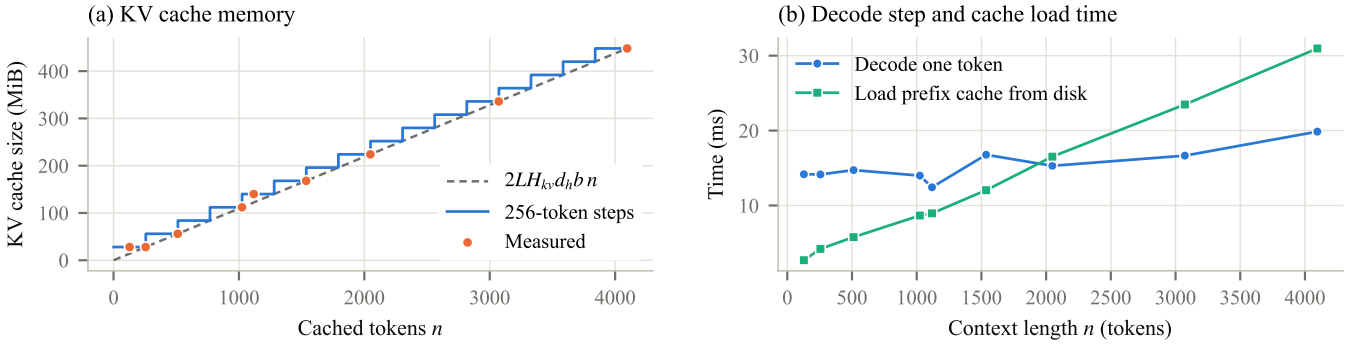


Figure 5: (a) Measured KV size (dots) matches eq. (5) exactly, including the 256-token allocation steps. (b) Decode time per token rises slowly with context; loading a prefix cache that is already in the OS page cache takes 3–31 ms.

Table 5: Multi-turn routing (*new in revision 2*): 10 sessions × 10 held-out requests, history ≤3 turns, all five configurations interleaved per session in rotated order. Absolute times are higher than in table 3 because of background load during the run; compare rows. Pref.: tokens prefilled per turn. Exact: decision vs. the single-turn label.

History / cache	TTFT	Turns 1–4	Turns 5+	Pref.	Exact
None / prefix	96	96	95	21	92
Sliding / prefix (default)	254	192	295	103	90
Sliding / + conversation	235	138	300	87	90
Block / prefix	208	181	227	82	87
Block / + conversation	**147**	136	**154**	42	87

token is identical at all nine lengths. End-to-end, greedy outputs with and without the cache were identical in 20 of 20 requests in both cached modes. Caching therefore changes latency, not behaviour.

8.5 Conversation-level reuse

The prefix cache stops at the system prompt, so a request with three turns of history still prefills about 100 tokens and its TTFT roughly triples. R2 can therefore keep the KV state of each routing prompt and, on the next turn, reuse the longest token prefix it shares with a stored one. The store is mlx-lm's `LRUPromptCache`, a token trie that hands out copies, which are cheap because MLX arrays are copy-on-write (about 3 ms per turn). Generated tokens are trimmed off before storing, because Qwen3 renders earlier assistant turns without the empty think block, so each turn still prefills the previous reply and the new request. Only live conversation turns are stored; planner clauses, evaluation and warm-up are not, so they cannot evict the entry.

Table 5 gives three results. First, reuse does not change behaviour: all 100 intents and graded parameters were identical with and without it, and the only differences were free-text refusal reasons (3 and 2 of 100), as section 8.4 predicts. Second, with R2's sliding window, reuse helps only while history grows (turns 2–4, −28%). Once the window is full, the oldest turn is dropped and every later token shifts position. Nothing beyond the system prefix then matches, and turns 5+ are unchanged. Third, a *block* window, which keeps only the latest turn when the window overflows, keeps the prefix stable and gives a flat ≈150 ms (−42% overall, −48% after turn 4). The shorter history, however, cost 3 of 100 exact decisions, among them "make it better" routed to a (confirmation-gated) `remember_preference`. Without any history, routing is 2.7× faster than the default and no less accurate on these single-turn labels. The benefit of history for follow-ups ("open it", "why did that fail?") is not measured

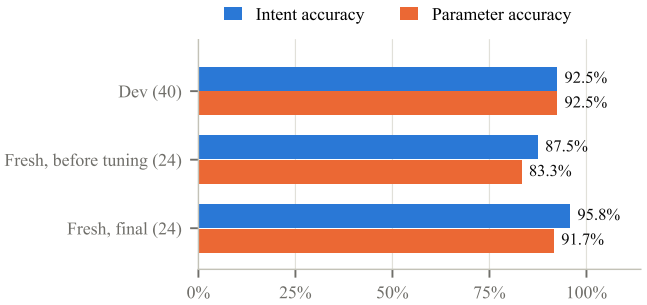


Figure 6: Intent and parameter accuracy on the dev set and on the fresh test set before and after the final prompt revision.

Table 6: Accuracy with 95% Wilson intervals. All runs: 0% malformed, 0% fallback.

Set	n	Intent acc.	Param. acc.
Dev	40	92.5 [80.1, 97.4]	92.5 [80.1, 97.4]
Fresh, before revision	24	87.5 [69.0, 95.7]	83.3 [64.1, 93.3]
Fresh, final	24	**95.8** [79.8, 99.3]	**91.7** [74.2, 97.7]

by this set. Conversation reuse therefore ships *off* by default (one stored entry costs 147 MB), and the history policy, not the cache, is the larger lever.

8.6 Accuracy

Table 6 and fig. 6 report accuracy. On the fresh set R2 chooses the right intent for 23 of 24 requests and the right parameters for 22. The prompt revision that sharpened the boundaries between `general_help`, `answer_question`, `clarify_request` and `unsupported` raised fresh-set intent accuracy from 87.5% to 95.8%. Because the fresh set predates the revision, this is evidence of generalisation, although the confidence intervals are wide at these sample sizes. Median decision latency is 296 ms on the dev set (90th percentile 359 ms) and 309 ms on the fresh set (fig. 7). These runs used the original configuration (intent-name mask, unknown keys dropped). The current default (schema grammar, unknown keys rejected) gives exactly the same accuracy on both sets, with 0% malformed output and a mean decision latency of 263 ms (dev) and 262 ms (fresh); see table 8.

8.7 Ablation: few-shot examples vs. constrained decoding

Table 7 and fig. 8 separate the two techniques. The 25 few-shot examples raise intent accuracy by 7.5 points and parameter accuracy by 15 points. Constrained decoding changes nothing on this set. Its mask only governs the intent *name*, and the single malformed output without examples placed a parameter at the top level of the JSON object, outside the mask's scope. Its value is a guarantee rather than an accuracy gain: no output can name a non-existent action. The cost of the examples is prompt length (675 of 1,118 tokens), which is exactly what the

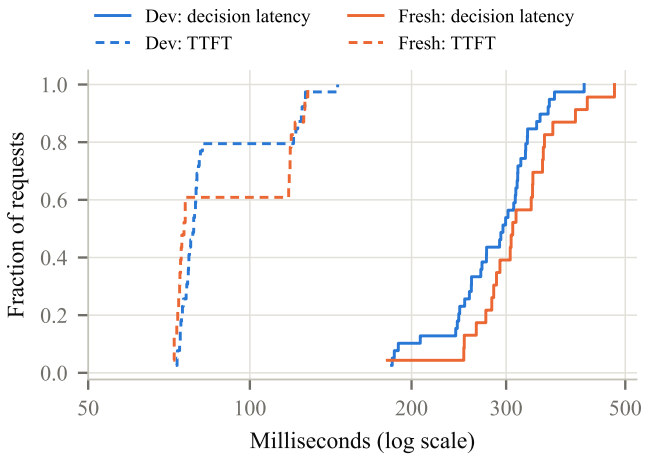


Figure 7: Cumulative distribution of TTFT and decision latency over the dev and fresh sets (the first request, which includes GPU kernel compilation, is excluded).

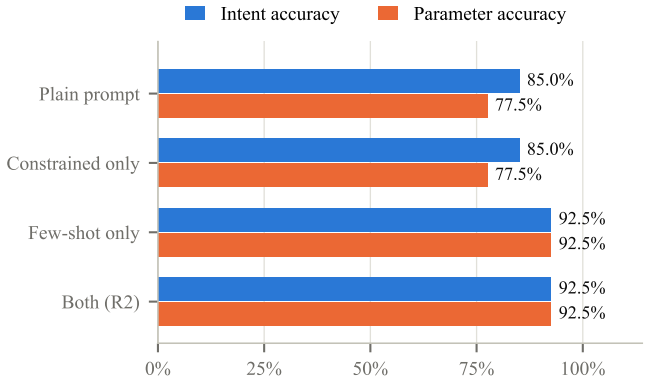


Figure 8: Ablation on the dev set. Few-shot examples account for the whole gain.

prefix cache makes free: without caching, few-shot prompting would add roughly 0.9 s to every request (eq. (7)).

Strict validation and the schema grammar. Dropping unknown parameter keys hides model errors, so the current parser rejects them. Table 8 shows the cost: the model often adds a stray parameter, such as `git_status {"clean": ...}`, and rejecting these sends 5% of dev requests to the heuristic fallback, lowering dev accuracy from 92.5% to 87.5%. The intent-name mask cannot help, because every one of these failures is in the parameters. Constraining decoding to the full decision schema makes such outputs impossible to generate. It recovers all of the loss, with zero fallbacks on both sets, and is not slower: decode stays at 12.5–13.9 ms per token in every mode, because the mask is computed on the CPU in tens of microseconds. Without examples the grammar adds 12.5 points of intent accuracy on both sets, since it removes the malformed outputs that cause most of those errors. These are small sets: one case is 2.5 points on dev and 4.2 on fresh, so the differences are indicative rather than significant.

Table 7: Ablation on the dev set (40 cases), original configuration: intent-name mask, unknown parameter keys dropped.

Few-shot	Constrained	Intent	Params	Malformed
		85.0	77.5	2.5
	✓	85.0	77.5	2.5
✓		92.5	92.5	0.0
✓	✓	92.5	92.5	0.0

Table 8: Constraint modes under *strict* validation (unknown keys rejected). Intent / parameter accuracy in %; fallback and mean decision latency on dev.

FS	Constraint	Dev (40)		Fresh (24)		Fallb. / ms
		Int.	Par.	Int.	Par.	
✓	none	87.5	87.5	91.7	87.5	5.0 / 295
✓	intent	87.5	87.5	91.7	87.5	5.0 / 287
✓	**json**	**92.5**	**92.5**	**95.8**	**91.7**	**0.0 / 263**
	none	72.5	65.0	75.0	75.0	15.0 / 287
	intent	72.5	65.0	75.0	75.0	15.0 / 274
	json	85.0	77.5	87.5	87.5	0.0 / 208

8.8 Multi-step planning

Table 9 compares planners on a dev set of 61 multi-step requests (connectors, a bare "and", three to five steps, single questions that contain "and then", and requests with a destructive or out-of-scope part that must be refused) and on a fresh set of 30 written before the clause planner's options were implemented. The options raise exact-plan accuracy on the fresh set from 66.7% to 93.3% while still refusing every must-refuse request; the price is about 230 ms of extra routing calls. The one-shot model planner is faster but produces well-formed, wrong plans that no validator can catch: it drops steps, copies few-shot parameters, and, most importantly, *silently omits* the part of a request it should refuse ("show me the diff then change it" becomes just the diff) instead of rejecting the plan. The clause planner's remaining errors are single-intent routing mistakes, such as "commit everything" routed to `git_status`.

8.9 Error analysis

All four intent errors (figs. 9 and 10) share one pattern: vague commands such as "do the thing", "make it better", "check that" and "change that thing" are classified as `unsupported`, often with the reason "destructive actions are not allowed", instead of prompting a clarifying question. The model conflates *unclear* with *unsafe*. This failure is benign, because a refusal executes nothing, but it is the main remaining accuracy gap. The single parameter error maps "list the yaml files" to `*.yml` rather than `*.yaml`, a defensible reading that an extension-alias table would resolve. During development we also found and fixed a normalisation bug in which `*swift*` matched compiler caches under `.build/`; file search now skips build directories.

Table 9: Planners on multi-step requests (plans validated, not executed). Fresh: 30 requests written before the and-split and question guard existed. Rej.: share of must-refuse requests refused.

Planner	Dev (61)	Fresh (30) [95% CI]	Rej.	p50
Clause	75.4	66.7 [48.8, 80.8]	100	488 ms
Clause + and-split + guard	**93.4**	**93.3** [78.7, 98.2]	100	720 ms
Model (one shot)	47.5	40.0 [24.6, 57.7]	0	310 ms

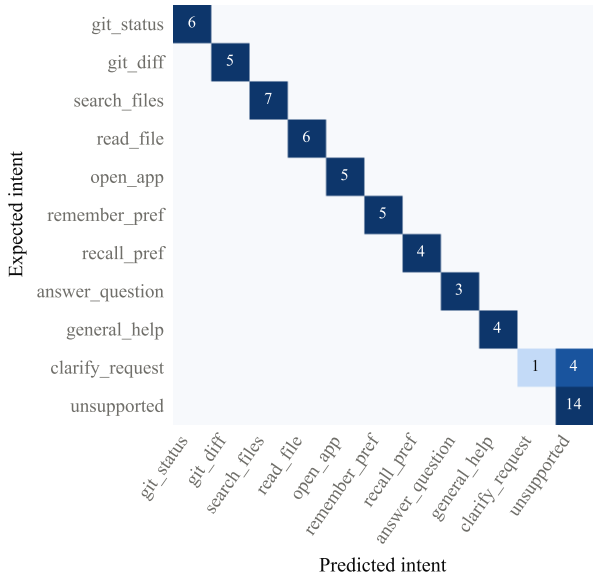


Figure 9: Confusion matrix over dev and fresh sets (64 cases). All intent errors are vague commands labelled `unsupported` instead of `clarify_request`.

8.10 Voice pipeline

Whisper-small transcribed all eight synthetic utterances without word errors in a median 281 ms (real-time factor 0.17), and Kokoro-82M synthesised replies in a median 301 ms (real-time factor 0.084, fig. 11). A spoken request therefore flows through recognition (≈0.28 s), decision (≈0.30 s) and the start of synthesis (≈0.25 s for a short reply), well under a second, with every stage on device.

9 Discussion

Why narrow works. Narrowness turns the hardest parts of an LLM assistant into engineering problems with clean answers. A fixed prompt makes prefix caching exact and nearly free; a closed intent set with typed parameters makes constrained decoding small and sound; and typed decisions make authorisation a deterministic policy rather than a property of model text. The resulting assistant is fast because it reads the long part of its prompt once, and safe because the model has no path to execution that the policy does not see.

Caching makes few-shot prompting affordable. The ablation and the cost model connect: few-shot examples provide

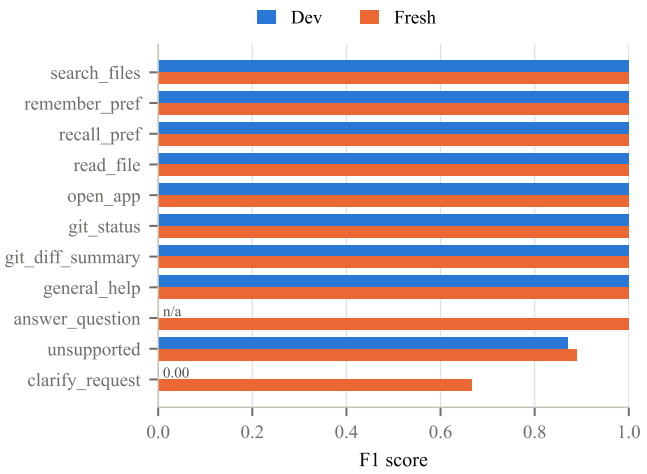


Figure 10: Per-intent F1. `answer_question` does not occur in the dev set.

the accuracy, and the prefix cache removes their latency cost. Without the cache, each example token costs about 1.4 ms on every request; with it, examples are paid for once at start-up (or never, with a warm disk cache). This suggests a general design rule for on-device assistants: put as much of the specification as possible into a stable prefix, and keep the variable part short.

Limits of the design.

- **Small evaluation sets.** With 40 and 24 cases, one case moves accuracy by 2.5–4.2 points, and the intervals in table 6 overlap.
- **Single machine, synthetic speech.** Timings come from one M4. The speech test uses synthesised rather than recorded, accented or noisy speech.
- **Free-form answers.** `answer_question` replies from a 1.7B model are short and sometimes imprecise. They are labelled as answers and never executed.
- **History reuse needs a stable prefix.** Conversation-level reuse is implemented (section 8.5), but with a sliding history window it only helps the first turns, and the block window that keeps it effective cost some accuracy. Neither is on by default.
- **Grammars choose values.** A schema grammar always produces *some* legal string, so constraining allow-listed values (app names) turns an unknown request into a wrong one. R2 leaves such values free and lets the gate refuse them.
- **Same-user trust boundary.** The API token stops browsers and other accounts, not processes running as the user (section 6.3).
- **Multi-step planning.** The planner comparison uses 61 dev and 30 fresh requests, so its intervals are wide, and the clause planner inherits every single-intent routing error.

Threats to validity. The dev set influenced prompt design, so its accuracy is optimistic; the fresh set mitigates but does not remove this, since the final revision was guided by general failure categories observed on the dev set. Benchmarks ran on a machine with other applications open; we report medians and show per-request spread, and a run-time watchdog aborted runs if free memory dropped below 25%. The cache-mode and scaling measurements were taken before the content-derived fingerprint was introduced; a re-run with it reproduced the 71 ms cached TTFT and identical greedy outputs across cold build, disk load and no cache, since the new fingerprint reads link targets rather than weight files. The constraint-mode comparison (table 8) uses the same single model, machine and evaluation sets as the rest of the paper.

Ethics and privacy. All audio, text and preferences stay on the device; no account or network service is required after the one-time model download. Preferences are stored only after explicit confirmation, and the assistant cannot read common secret files. The run history of multi-step requests keeps only metadata and never file contents, diffs or answers.

10 Future Work

1. **History policy:** a multi-turn set with real follow-ups (“open it”, “why did that fail?”) to decide how much history routing needs, and whether a block window with conversation reuse [8] is worth its accuracy cost. A further step would be to prefill the next turn’s rendered reply while the answer is spoken.
2. **KV quantisation:** measure 8- and 4-bit caches [16, 17], expected to matter only for longer contexts.
3. **Multi-step evaluation:** a multi-step set written by someone other than the authors, and recorded spoken multi-step requests.
4. **Clarification:** a contrastive set of vague vs. destructive requests and LoRA fine-tuning [38] on a larger synthetic dataset.
5. **Decode speed:** speculative decoding [39] for longer spoken answers.
6. **Larger studies:** more requests, recorded speech, several Apple chips and other small models.

11 Conclusion

R2 shows that a small local model can feel like a native voice assistant when the system around it is designed for a narrow task. A persistent prefix KV cache turns a 1.5 s prompt into a 71 ms response start with identical outputs. Its behaviour follows from a few lines of theory: prefix invariance, the KV memory formula and a linear-plus-quadratic cost model. Few-shot examples deliver 95.8% intent accuracy on unseen requests, a schema grammar keeps that accuracy under strict validation, and a fail-closed policy keeps every action, including each step of a multi-step request, under the user’s control.

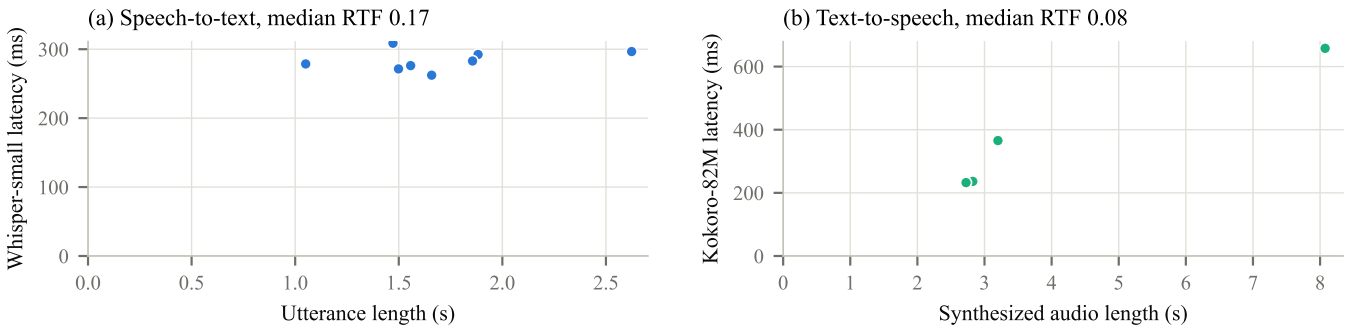


Figure 11: On-device speech latency. (a) Whisper-small transcribes 1–2.6 s utterances in about 0.28 s. (b) Kokoro-82M synthesises speech about 12× faster than real time.

Reproducibility

All numbers in this paper are produced by scripts in the repository and stored in `artifacts/`:

```
uv sync && uv run python -m pytest          # 249 tests
uv run python training/bench_kv.py           # Table 3, Fig. 3
uv run python training/bench_scaling.py      # Table 4, Figs.
    4-5
uv run python training/bench_conversation.py # conversation
    reuse table
uv run python training/evaluate.py --backend mlx [--eval data/
    eval_fresh.jsonl]
uv run python training/evaluate.py --backend mlx --no-few-shot
    --no-constrain
uv run python training/ablation.py           # constraint modes
    , Table 7
uv run python training/eval_multistep.py --backend mlx [--data
    data/eval_multistep_fresh.jsonl]
uv run python training/bench_voice.py        # Fig. 11
uv run python paper/make_figures.py          # all figures
```

Author Contributions

Siavash Shokrtalab (full-stack engineering): system architecture, Python core and HTTP API, SwiftUI macOS client, voice pipeline integration, tooling and testing. **Alireza Panahdoost** (research and machine learning): research questions, KV cache and constrained-decoding methodology, theoretical analysis, benchmark and evaluation design, error analysis. Both authors wrote and approved the manuscript.

Revision History

Revision 2 follows an external review of the code and of revision 1. It adds token, Host and Origin checks to the local API and binds confirmations to a single-use, expiring id (section 6.3); derives the cache fingerprint from content and validates every cache layer, with atomic writes and on-disk eviction (section 6); rejects unknown parameters and adds schema-constrained decoding with its ablation (sections 4.5 and 8.7); hardens the file tools against links and races; adds bounded multi-step requests with a client view (section 5.4); and measures conversation-level KV reuse (section 8.5). Revision 1 numbers are unchanged; new measurements are marked as such.

Acknowledgements

We thank the developers of MLX, mlx-lm, Qwen3, Whisper and Kokoro for releasing their work openly. Parts of the implementation and of this manuscript were prepared with the help of an AI coding assistant; the authors are responsible for all content.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL `https://arxiv.org/abs/1706.03762`. Peer-reviewed.

[2] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023. URL `https://arxiv.org/abs/2211.05102`. Peer-reviewed.

[3] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019. Preprint.

[4] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL `https://arxiv.org/abs/2305.13245`. Peer-reviewed.

[5] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. Preprint.

[6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL `https://arxiv.org/abs/2205.14135`. Peer-reviewed.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings*

of the 29th ACM Symposium on Operating Systems Principles (SOSP), 2023. doi: 10.1145/3600006.3613165. URL https://arxiv.org/abs/2309.06180. Peer-reviewed.

[8] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2312.07104. Peer-reviewed.

[9] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. URL https://arxiv.org/abs/2311.04934. Peer-reviewed.

[10] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2025. URL https://arxiv.org/abs/2405.16444. Peer-reviewed.

[11] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM Conference*, 2024. URL https://arxiv.org/abs/2310.07240. Peer-reviewed.

[12] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, 2023. Software; prompt-cache and slot-save features.

[13] Anthropic. Prompt caching with Claude. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching, 2024. Industry documentation.

[14] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.17453. Peer-reviewed.

[15] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.14048. Peer-reviewed.

[16] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2402.02750. Peer-reviewed.

[17] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2401.18079. Peer-reviewed.

[18] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023. Preprint.

[19] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL https://arxiv.org/abs/2305.13971. Peer-reviewed.

[20] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and efficient structured generation engine for large language models. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025. URL https://arxiv.org/abs/2411.15100. Peer-reviewed.

[21] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. In *Proceedings of the ACM on Programming Languages (PLDI)*, 2023. doi: 10.1145/3591300. URL https://arxiv.org/abs/2212.06094. Peer-reviewed.

[22] Guidance AI. llguidance: Super-fast structured outputs, 2025. URL https://github.com/guidance-ai/llguidance. Software, version 1.9.

[23] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2302.04761. Peer-reviewed.

[24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629. Peer-reviewed.

[25] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2305.15334. Peer-reviewed.

[26] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2005.14165. Peer-reviewed.

[27] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.17323. Peer-reviewed.

[28] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. URL https://arxiv.org/abs/2306.00978. Peer-reviewed.

[29] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. https://github.com/ml-explore/mlx, 2023. Software; mlx 0.32.4 and mlx-lm 0.32.2 used here.

[30] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2212.04356. Peer-reviewed.

[31] hexgrad. Kokoro-82M: An 82m-parameter text-to-speech model. https://huggingface.co/hexgrad/Kokoro-82M, 2024. Model card; MLX conversion mlx-community/Kokoro-82M-bf16.

[32] Yinghao Aaron Li, Cong Han, Vinay S. Raghavan, Gavin Mischler, and Nima Mesgarani. StyleTTS 2: Towards human-level text-to-speech through style diffusion and adversarial training with large speech language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.07691. Peer-reviewed.

[33] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. In *Neurocomputing*, volume 568, 2024. doi: 10.1016/j.neucom.2023.127063. URL https://arxiv.org/abs/2104.09864. Peer-reviewed (journal).

[34] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM 52(4):65–76, 2009. Peer-reviewed (journal).

[35] Hugging Face. Understand caching: the Hugging Face Hub cache layout, 2025. URL https://huggingface.co/docs/huggingface_hub/how-to-cache. huggingface_hub documentation.

[36] Model Context Protocol. DNS rebinding protection for local servers (GHSA-w48q-cv73-mx4w), 2025. URL https://github.com/modelcontextprotocol/typescript-sdk/security/advisories/GHSA-w48q-cv73-mx4w. Security advisory, TypeScript SDK.

[37] Chrome for Developers. Local network access: permission prompts for requests to local devices, 2025. URL https://developer.chrome.com/blog/local-network-access.

[38] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2106.09685. Peer-reviewed.

[39] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2211.17192. Peer-reviewed.

A System Prompt Excerpt

```
You are R2, a narrow, local macOS developer assistant (like Siri, but only for developer tasks in the
user's active workspace).

Turn the user's request into exactly ONE JSON object and nothing else:
{"intent": "<intent>", "parameters": {...}}

Intents (parameters) - when to use:
- general_help {} - ONLY when the user asks about R2 itself: what you can do / how to use you
- git_status {} - branch, repo status, uncommitted or changed files
- git_diff_summary {} - summary or size of the diff / what changed in the code
- search_files {"pattern": "<glob, e.g. *.py, *test*, README*>"} - find, list or locate files by name or type
- read_file {"path": "<relative file path>"} - show, read, print or display the contents of a specific file
- open_app {"app_name": "<one of the approved apps>"} - launch or open an application
- remember_preference {"key": "<snake_case key>", "value": "<value>"} - user tells you a personal preference ...
- recall_preference {"key": "<snake_case key>"} - user asks about their OWN previously saved preference ...
- answer_question {"question": "<the user's question>"} - a software-development knowledge question ...
- clarify_request {"question": "<short follow-up question>"} - a vague command whose target is unclear ...
- unsupported {"reason": "<short reason>"} - chit-chat, trivia, math, destructive or out-of-scope requests
... approved apps, rules, and 25 worked examples (675 tokens) follow.
```

B Local HTTP API

Every route except `/health` requires `Authorization: Bearer <token>`; see section 6.3 for the Host, Origin and content-type rules.

Method	Path	Purpose
GET	`/health`, `/status`, `/voices`	liveness (no token); model and KV cache state; voice list
POST	`/ask` {`text`, `confirmation_id`?, `allow_runs`?}	text request → reply, speech, intent, metrics, confirmation id; may start a run
POST	`/confirm` {`confirmation_id`}, `/cancel`, `/reset`	approve the pending action by id; cancel; clear history
POST	`/ask_audio` (WAV, ≤16 MB)	Whisper transcription, then `/ask`
POST	`/tts` {`text`, `voice`, `speed`}	Kokoro speech as WAV (speed 0.5–2.0)
POST	`/run` {`text`}	plan and start a multi-step run
GET	`/runs/<id>`	poll a run (never returns the confirmation id)
POST	`/runs/<id>/confirm`, `/cancel`	approve the paused step by id; cancel the run

C Evaluation Set Composition

Intent	Dev	Fresh	Intent	Dev	Fresh
`git_status`	4	2	`recall_preference`	2	2
`git_diff_summary`	3	2	`answer_question`	0	3
`search_files`	5	2	`general_help`	3	1
`read_file`	4	2	`clarify_request`	3	2
`open_app`	3	2	`unsupported`	10	4
`remember_preference`	3	2	**Total**	**40**	**24**